SHRINCS Parameter Search Methodology

Blockstream Research

May 19, 2026

Contents

1 Introduction

Post-quantum signatures are rapidly evolving and getting better, while the day, when a quantum computer will be able to break the classic signature schemes, is closer that we might think. This creates the need to transion to post-quantum schemes, especially in the systems where the signture algorithms are the main instrument, for example, Bitcoin. In the case of the Bitcoin, hash-based signature scheme (SPHINCS+ or SLH-DSA) is the most compatible one among all quantum-resistant algorithms. While the NIST standard stateless hash-based signature scheme parameter choice making the it suboptimal to implement in the Bitcoin, we still can optimize the algorithm to be more efficient. This could be achived through optimizing the algorithm structure or only changing construction parameters. For the first case, we need to make sure that the modifications are proven to be secure which needs to pass the test of time with enough research done. The more reasonable and safe option is to change parameters to achive smaller signature size and smaller signing/verification costs. Fortunely for us, we have enough room to play with the paraters to obtain signatures that sufficient for the Bitcoin.

In this document we will show and explain the methodology of searching parameters for SHRINCS stateless part (SHL-DSA architecture for 2^{40} signatures).

2 Parameters Description

2.1 Initial Parameters

As mentioned in the introduction, our objective is to find such parameters that will be better than SLH-DSA. For the Bitcoin we require no more than 128-bit security. If we put the security cap higher this will lead to bigger signature size and costs, which is not preferable. Another parameter that sets the further foundation for the parameter search is the number of signatures, namely q_s. In SLH-DSA-128s version, the number of signatures is set to be $q_s = 2^{64}$. While

parameters for standardized SLH-DSA are created for general purpose usage, the number of required signatures in Bitcoin is lower. Setting $q_s = 2^{40}$ is enough for our purposes, which will decrease the size and the computational cost.

2.2 SLH-DSA Parameters

The basic architecture of SLH-DSA relies on a hyper-tree of extended Merkle signature schemes (XMSS) to authenticate underlying one-time and few-time signature keys. We formally parameterize each candidate configuration using the tuple (h, d, k, a, w), where each variable controls a distinct geometric trade-off within the overall structure.

Hyper-Tree Geometry (h and d) The hyper-tree configuration determines the capacity and layer traversal costs of the signature scheme.

- h represents the total height of the multi-layered hyper-tree. This variable dictates the maximum number of few-time signature instances supported at the base layer, yielding a total capacity of 2^h unique leaf nodes.
- d defines the number of independent tree layers comprising the hyper-tree.

To ensure symmetrical construction across all internal boundaries, our search pipeline strictly mandates that the total height h must be perfectly divisible by the layer count d. Consequently, each individual XMSS tree layer operates with a uniform height denoted as h':

$$h' = \frac{h}{d}$$

Adjusting the ratio between h and d governs the balance between signature size and key generation speed. Minimizing layers (d) reduces the total number of intermediate public keys and authentication paths included in the signature payload, saving valuable bytes. However, this exponentially increases the height h' of each constituent tree layer, requiring significantly more hash operations to compute the root node during key generation.

Few-Time Signature Layer (k and a) The lowest layer of the hyper-tree authenticates the Forest of Random Subsets (FORS) few-time signature scheme, which directly signs the message digest.

- k represents the number of independent, parallel Merkle trees operating within the FORS layer.
- a defines the height of each individual FORS tree. This parameter dictates that each constituent tree contains exactly $t = 2^a$ leaves.

Increasing k and a strengthens the scheme against multi-target subset forgery attacks, allowing the structure to safely support larger capacities. However, expanding these dimensions directly increases the byte footprint of the appended authentication paths.

One-Time Signature Base (w) Internal tree roots are signed using the Winternitz One-Time Signature Plus (WOTS+) scheme. The base parameter w defines the underlying digit representation used to split the message digest during signing. Our framework evaluates candidate configurations across standard baseline settings where $w \in \{16, 32, 256\}$.

Selecting a larger base value significantly shortens the required number of parallel hash chains (l), which reduces the overall WOTS+ signature payload size. This spatial efficiency trades against local compute cycles: larger base parameters require significantly longer individual hash chains, resulting in higher execution costs during public key generation and signature creation.

We set the ranges as follows: $(h, d, k, a, w) \in \{40, ..., 52\} \times \{2, ..., 26\} \times \{2, ..., 25\} \times \{8, ..., 21\} \times \{16, 32, 256\}$.

3 Search Methodology

3.1 Initial Search Conditions

Searching for the best parameters require defining conditions that the best parameters set must to satisfy. Consequently, the conditions must rely on some metrics, which depend on the parameters. We use the following metrics:

- Signature size: the size of the signture in bytes;
- Key generation cost: the number of SHA-256 compressions computations during key generation phase;
- Signing cost: compressions evaluated during signing phase;
- Verification cost: the number of compressions during verification phase;
- compression per byte: the ratio between verification cost to the signature size.

Considering the specifics of the Bitcoin, we require to have size smaller than in the SLH-DSA. Note, we previoulsy set the security level must equal to 128-bits, which is also condition that must to be satisfied.

We choose SLH-DSA-128s as the baseline for the search, which gives us an intuitive way to fairly compare the metrics between custom parameters and standartized SLH-DSA.

3.2 Search Strategy

With initialized search space, we start by running through all parameter pairs (h, d, k, a, w) that satisfy security bound and signature size smaller than SLH-DSA. Applying this conditions reduce search space from 41,328 candidates to 9229. But the space of posible parameters is still big to traverse, so we need to apply other conditions to make the possible parameters set smaller.

Let's introduce the following filters to defined metrics:

- Bound on signature size (input: max_size): size $\leq$ max_size;
- Bound on key generation cost (input: X): $\text{cost}_{\text{kg\_custom}} \leq X \cdot \text{cost}_{\text{kg\_std}}$;
- Bound on signing cost (input: X): $\text{cost}_{\text{sig\_custom}} \leq X \cdot \text{cost}_{\text{sig\_std}}$;
- Bound on verification cost (input: X): $\text{cost}_{\text{ver\_custom}} \leq X \cdot \text{cost}_{\text{ver\_std}}$;
- Bound on compression per byte (input: X): $\text{cost}_{\text{ver\_custom}}/\text{size} \leq X$

where X is arbitrary positive real number, and max_size is the positive integer value.

We can apply follwoing filters in different combinations, with different values used to bound the metrics to get the desired result. Let's consider the following cases to investigate how the filters setup affects the set of suitable parameters.

Absolute optimal single metric This is case, where we try to find the smallest metric while keeping only initial conditions. The bounds on other metrics are unlimited. The results shown in the table.

Table 1: Absolute Global Extremes

Award Category	Tuple (h, d, k, a, w)	Size (B)	vs. Std	Keygen (C)	KG/Std	Sign (C)	SG/Std	Verify (C)	ρ (C/B)
STANDARD Baseline	$(63, 7, 14, 12, 16)$	7872	+0.0%	2.93×10^5	1.00x	2.28×10^6	1.00x	2387	0.30
Smallest Signature	$(50, 2, 6, 20, 256)$	3424	−56.5%	1.55×10^{11}	5.29×10^5x	3.10×10^{11}	1.36×10^5x	4953	1.45
Fastest Keygen	$(40, 10, 8, 20, 32)$	7680	−2.4%	1.40×10^4	0.05x	3.37×10^7	14.78x	4673	0.61
Fastest Signing	$(40, 8, 23, 8, 32)$	7440	−5.5%	2.80×10^4	0.10x	2.47×10^5	0.11x	3888	0.52
Fastest Verification	$(50, 2, 7, 17, 16)$	3968	−49.6%	1.92×10^{10}	6.55×10^4x	3.84×10^{10}	1.68×10^4x	894	0.23
Best Density Ratio (ρ)	$(50, 2, 23, 15, 16)$	7840	−0.4%	1.92×10^{10}	6.55×10^4x	3.84×10^{10}	1.68×10^4x	1366	0.17

From Table 1, we can see that the results for each metric is minimized as possible, but with the cost of other metrics being too big, which makes this parameters impractical to use. Let's consider strategy to find the suitable parameters.

Balanced Metrics If we ask a question, how the size set of parameters change, if we use common value X across all 4 metrics related to it? How the number of candidates changes depending on choice of filter input? What is the smallest X that we can set to obtain a few candidates? We apply the result for different values of X, and obtain the following results:

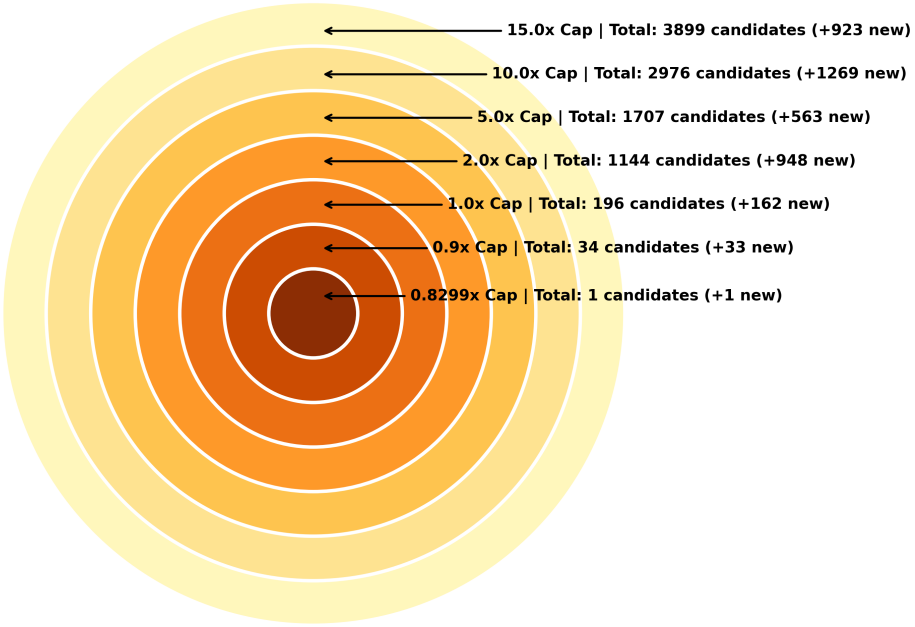

Figure 1: Global Minimax Expansion: Concentric Euler diagram illustrating the volume of candidates unlocked at expanding uniform multiplier thresholds.

The bigger the multiplier, the bigger numer of candidates we get through the search process. And the oppossite is also true, with smaller X, the number of candidates becomes smaller. The smallest value of multiplier is $X \approx 0.8299$, and the results presented in the table 2

Even if it is balanced metric, sometimes there is need in tradeoffs and searching for smaller metric values. Below we explain and visualize the strategy to find such candidates.

Optimal Search Strategy Every candidates can be related to the some tuple of metrics value. The search for optimal candidate could be represented as finding the closest point to the origin in five dimentionl space (finding the shortest vector). The issue is that we cannot visualize the such high dimentional space, but we can analyse the 2D projections of 5 dimentional space. We could create 10 graphs of every metrics pairs, but It is quite complicated to analyse. The

Table 2: Performance Comparison between the Standard Baseline and the Balanced Candidate ($X = 0.8299$)

Metric	**STANDARD Baseline**	**Optimal Candidate**	**Relative Multiplier (X)**
Parameters (h, d, k, a, w)	$(63, 7, 14, 12, 16)$	$(40, 5, 21, 12, 16)$	–
Signature Size (Bytes)	$7,872$	$7,840$	$0.9959\times$
Key Generation Cost	$292,862.0$	$146,431.0$	$0.5000\times$
Signing Cost	$2,279,391.0$	$1,076,100.49$	$0.4721\times$
Verification Cost	$2,387.0$	$1,973.09$	$0.8266\times$
Hash Density Ratio (ρ)	0.303227	0.251648	$0.8299\times$

better option is the strategy, were we define the metrics that are important to us, and the rest we cound use the default filter, by setting $X = 1$. And to find the best candidate, we serach for the point that the the closest to the origin.

Let's see the the visualized example below, where we chose signature size and signing cost as important metrics, while other must be better than the baseline.

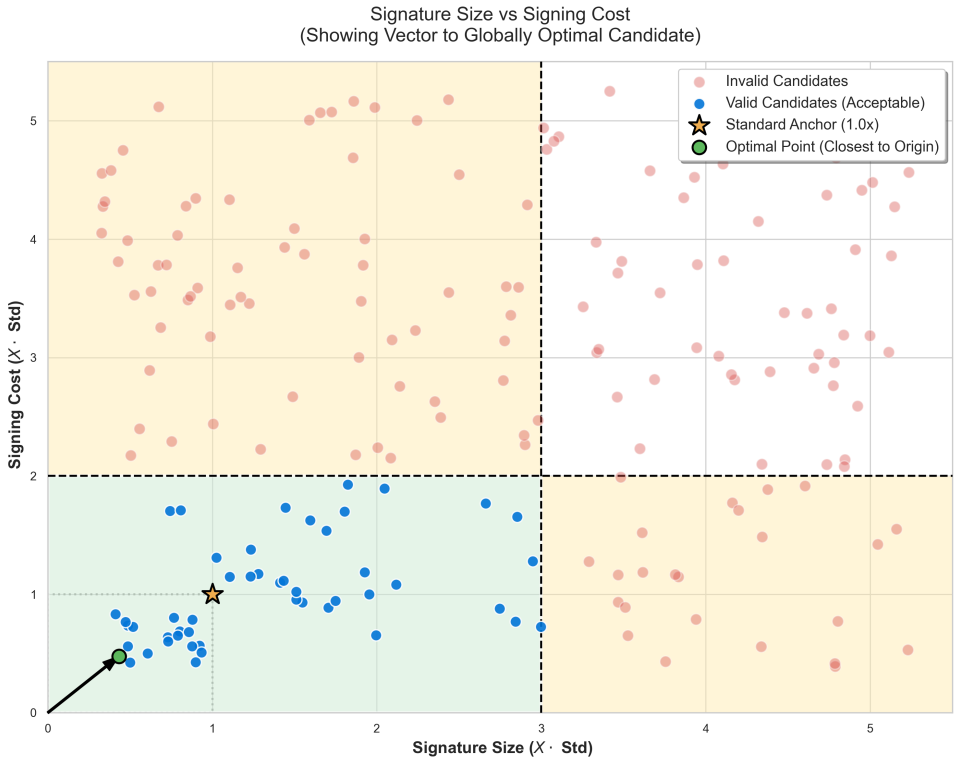

Figure 2: Visualization of Candidates with Prioritization of Size and Signing Cost Metrics.